University of Asia Pacific

Department of Computer Science and Engineering **CSE 437: Robotics Lab**

LAB REPORT

Experiment Number: 04

Experiment Title: Design and Implementation of a Self-Balancing Robot.

Submitted by :

Team Name : TriBots

Name : Tanjina Akter Nisa

Student ID : 22201086

Section : AB-2

Submitted to :

A S Zaforullah Momtaz
Assistant Professor,
Department of Computer Science and Engineering

Date of Submission : 07/03/2026

1. Experiment Name

Design and Implementation of a Self-Balancing Robot.

2. Objective

The objective of this experiment is to design and implement a self-balancing robot using an Arduino microcontroller and a motion sensor. The robot maintains its balance automatically by detecting its tilt angle and adjusting the speed and direction of the motors accordingly. This project demonstrates the practical application of control systems, sensor data processing, and motor control in robotics.

3. Apparatus / Hardware & Software Requirements

Hardware Components

- Arduino Uno microcontroller
- MPU6050 (Accelerometer + Gyroscope sensor)
- L293D Motor Driver IC
- Two DC Gear Motors
- Robot chassis with wheels
- Breadboard
- Jumper wires
- External power supply / battery (7.4V–12V)
- USB cable for programming

Software Requirements

- Arduino IDE
- Tinkercad for simulation
- Arduino libraries:
    - Wire.h
    - MPU6050 library

4. Circuit Diagram / Schematic

The circuit consists of three major parts:

1. Arduino Uno – controls the whole system
2. MPU6050 Sensor – detects tilt angle and motion
3. L293D Motor Driver – controls the DC motors

The MPU6050 sensor communicates with Arduino through I2C protocol, while the motor driver controls the motors using digital output pins.

Pin Configuration

MPU6050 Sensor → Arduino

MPU6050 Pin	Arduino Pin
VCC	5V
GND	GND
SDA	A4
SCL	A5
INT	D2

L293D Motor Driver → Arduino

L293D Pin	Arduino Pin
IN1	D8
IN2	D9
IN3	D10
IN4	D11
EN1	5V
EN2	5V
VCC1	5V
VCC2	Motor Power
GND	GND

Motors → L293D

Motor	L293D Pin	Motor
Left Motor	OUT1, OUT2	Left Motor
Right Motor	OUT3, OUT4	Right Motor

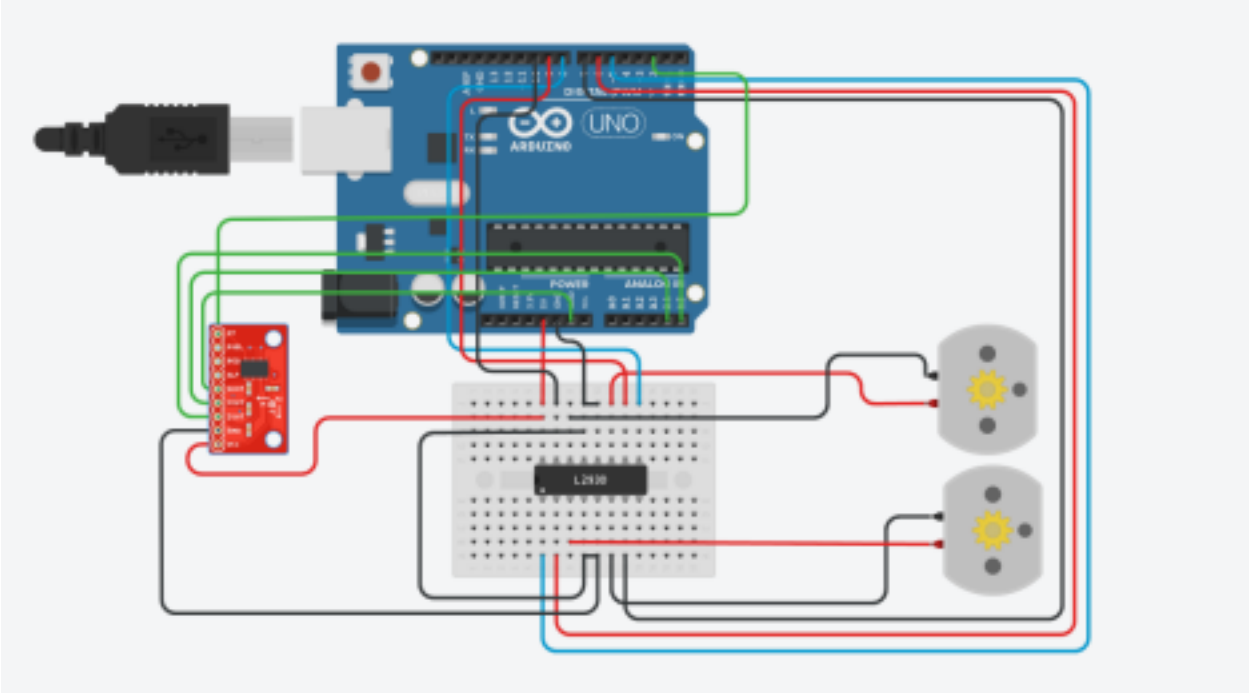

Fig 4.1: Circuit Diagram for Self-Balancing Robot.

5. Code / Assembly Program

```
#include <Wire.h>
#include <MPU6050.h>
MPU6050 mpu;

int motor1A = 8;
int motor1B = 9;
int motor2A = 10;
int motor2B = 11;

int16_t ax, ay, az;
int16_t gx, gy, gz;

void setup()
 {
 Serial.begin(9600);
 Wire.begin();
 mpu.initialize();
```

```
 pinMode(motor1A, OUTPUT);
 pinMode(motor1B, OUTPUT);
 pinMode(motor2A, OUTPUT);
 pinMode(motor2B, OUTPUT);
}
```



```
void loop()
 {
 mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

int angle = ay / 100;

 if(angle > 5)
 {
 forward();
 }
 else if(angle < -5)
 {
 backward();
 }
 else
 {
 stopMotor();
 }

 Serial.println(angle);
 delay(50);
}

void forward()
 {
 digitalWrite(motor1A, HIGH);
 digitalWrite(motor1B, LOW);
 digitalWrite(motor2A, HIGH);
 digitalWrite(motor2B, LOW);
}

void backward()
 {
 digitalWrite(motor1A, LOW);
 digitalWrite(motor1B, HIGH);
 digitalWrite(motor2A, LOW);
 digitalWrite(motor2B, HIGH);
}
```

```
void stopMotor()
 {
 digitalWrite(motor1A, LOW);
 digitalWrite(motor1B, LOW);
 digitalWrite(motor2A, LOW);
 digitalWrite(motor2B, LOW);
}
```


6. Output / Observations

During the experiment, the MPU6050 sensor continuously measures the tilt angle of the robot.

- When the robot tilts forward, the motors rotate forward to maintain balance.
- When the robot tilts backward, the motors rotate in the opposite direction.
- When the robot is in a stable upright position, the motors stop.

The tilt values are displayed in the Serial Monitor of the Arduino IDE.

7. Result

The self-balancing robot was successfully implemented using Arduino, MPU6050 sensor, and L293D motor driver. The robot was able to detect its tilt and adjust the motor movement accordingly to maintain balance.

8. Conclusion

In this experiment, a self-balancing robot was designed and implemented successfully. The MPU6050 sensor provided accurate tilt measurements, and the Arduino processed this data to control the motors through the L293D motor driver. This project demonstrates the integration of sensors, microcontrollers, and control algorithms in robotics. The system can be further improved using advanced control techniques such as PID control for more stable balancing.